

memory hierachy

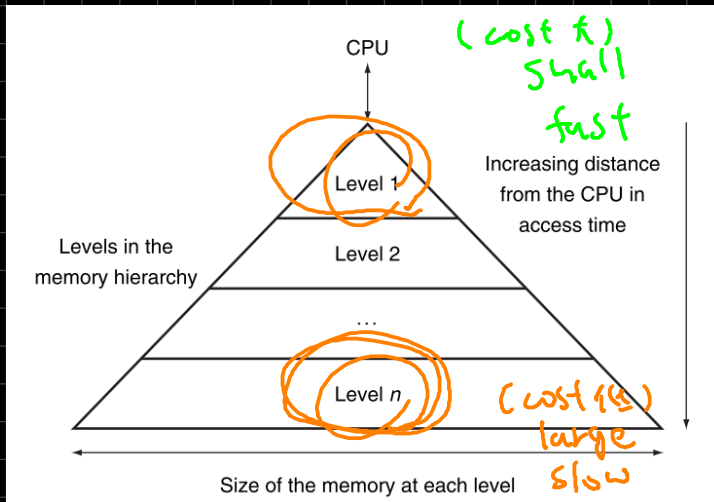
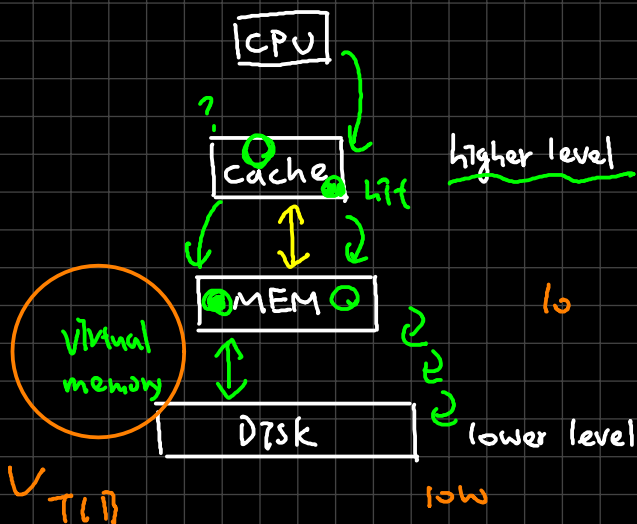


FIGURE 5.2 Every pair of levels in the memory hierarchy can be thought of as having an upper and lower level. Within each level, the unit of information that is present or not is called a *block* or a *line*. Usually we transfer an entire block when we copy something between levels.

goal: 希望 memory 越大 越快 越便宜

cache



情况

- (1) hit (在 higher level)
- (2) miss (higher level 找不到, 往 lower level 找)

Syntax

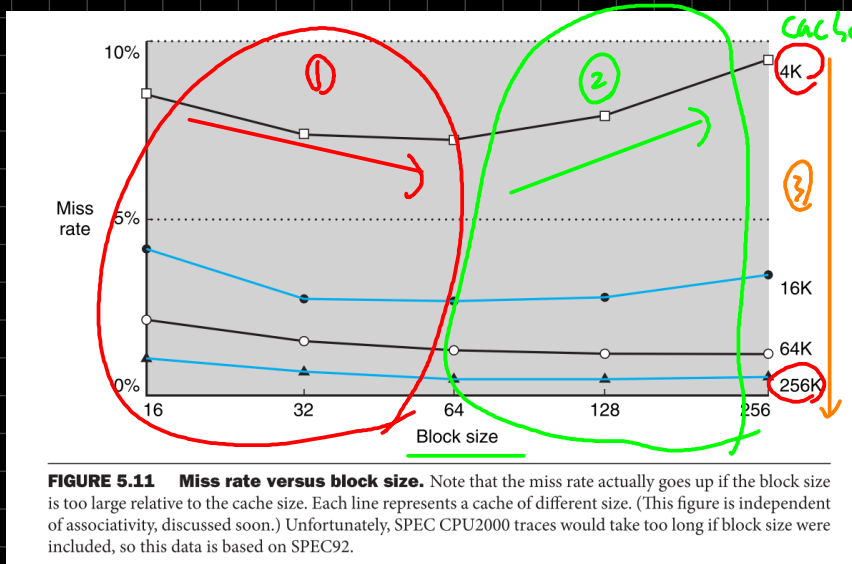
- (1) ↑ hit rate / miss rate ↓
(hit / miss to total access to 1/3 etc)

- (2) late fine (命中罰多入)
miss penalty (沒命中罰額外多入)

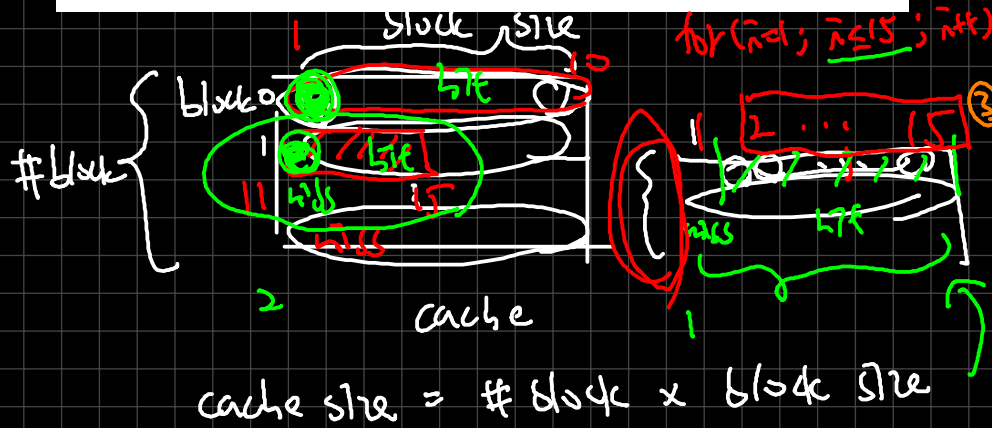
↓

↓ AMAT (Average Memory Access Time)

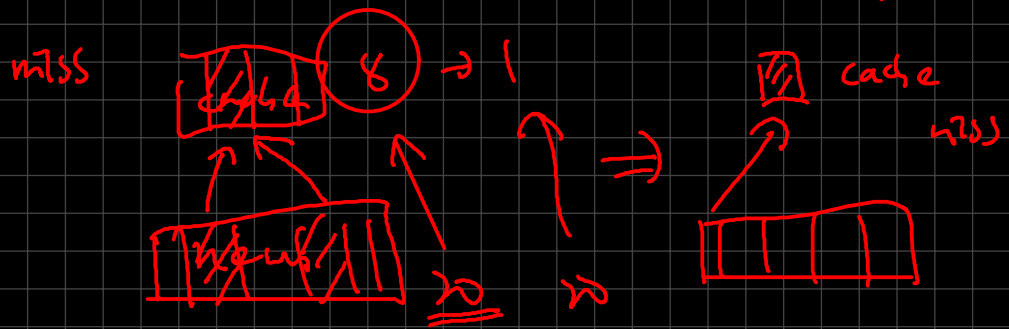
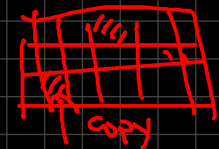
$$= \underbrace{\text{hit time}}_{L_{\text{hit}}} + \underbrace{\text{miss rate} \times \text{miss penalty}}_{L_{\text{miss}}}$$



- cache size
- ① cache size fixed
block size ↑, spatial locality ↑
(↓ - 存取鄰近) cache 的 data 變多
∴ miss rate ↓
 - ② cache size fixed
block size ↑, #block ↓
competition ↑
∴ miss rate ↑



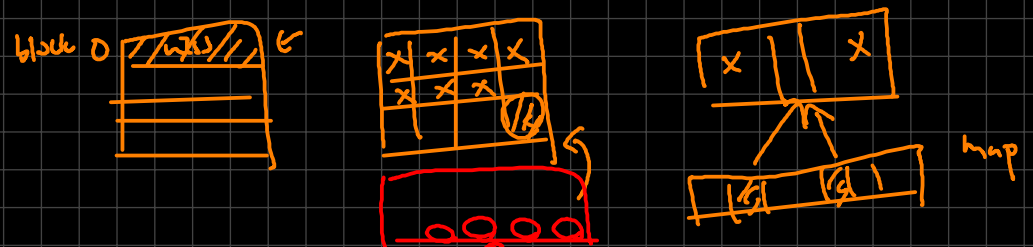
cache size 愈大
愈不受 block size 影響
(∵ 有足夠多的 block 而不會 miss)



3C miss model

$$3C = \text{Compulsory} + \text{Capacity} + \text{Conflict}$$

類型	中文	為什麼 miss ?	最快判斷
Compulsory miss	強制 / 冷啟動 miss	這個 block <u>第一次被存取</u>	<u>第一次看到</u>
<u>Capacity miss</u>	容量 miss	Cache 整體容量不夠	Cache 太小
<u>Conflict miss</u>	衝突 miss	Cache 明明還有空間，但多個 block 被迫搶同一個位置	位置打架



3. Consider the cache design.

- (a) Increasing block size can lower compulsory miss as well as miss penalty.
- (b) The reason why increasing cache size can lower capacity miss is the available block number in the cache increases.
- (c) Increasing block size within reasonable range can raise hit rate.
- (d) If memory stores an integer array in a row-based manner, it is more efficient to traverse this array by setting row variable in the outer loop and column variable in the inner loop.

(c) block size $\uparrow$, miss rate $\downarrow$ (hit rate $\uparrow$)

(d) row-based



```
for (i=1; i<3; i++) { // row  $\downarrow$ 
    for (j=1; j<3; j++) { // col  $\downarrow$ 
        A[i][j] = 0;
    }
}
```



low access
Spatial locality (neighbor)

Temporal = 等一下還會用「同一個」
Spatial = 等一下會用「旁邊的」

```
for (int i = 0; i < 100; i++)
    sum = sum + i;
```

```
int A[100];
for (int i = 0; i < 100; i++)
    sum += A[i];
```

$i=0$
 $A[0], A[1], A[2] \dots A[100]$
miss hit hit

(a) block size $\uparrow$, compulsory miss $\downarrow$

cache
block size = 50
 $A[0] \sim A[49]$

$\Rightarrow$ miss 把 100 的 data 一次裝進 cache, 利用 Spatial locality

block size = 1 word

read	0	miss
	1	miss
	2	miss
	3	miss

block size = 4 word

read	0	miss (load [0,1,2,3])
	1	hit
	2	hit
	3	hit

miss penalty $\uparrow$ ($\because$ block size 太小 $\therefore$ 從 mem 搬去 cache data 愈多)

1 block: mem $\xrightarrow{1 \text{ word}}$ cache
4 block: mem $\xrightarrow{4 \text{ word}}$ cache

(b) cache size = block size $\times$ # blocks $\uparrow$
fixed

cache size $\uparrow$, # blocks $\uparrow$, cache 有更多 blocks 而不會因為 裝不下 被 evict

$\therefore$ compulsory rate $\downarrow$

CPU → cache

cache operation

	read	write
hit	①	③
miss	②	④

①: read hit
→ 正常執行

②: read miss
→ CPU stall, 從 mem 抓 data 上 cache

③: write hit

→ write cache, 但 cache 存的是 mem 的 copy, 有 consistency 問題

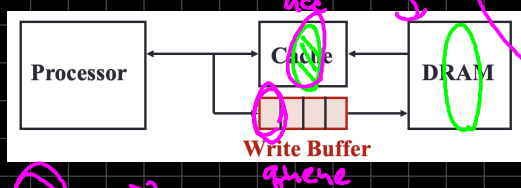
Solution

(i) write through: 同時 update cache & mem

(ii) write back: 直到最後一刻 (須被 evict 出 cache 時)

⇒ 用 dirty-bit 來判斷被寫

* 加速 = write buffer (batch)



②: dirty-bit = 1
cache

WB
5

④: write miss

→ CPU 要 write 的 data 不在 cache, 要去 mem 抓

Solution

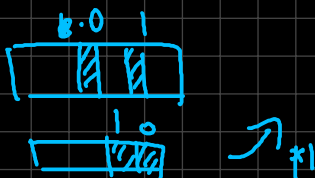
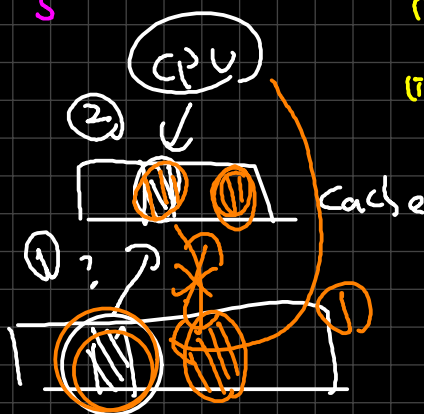
搭配: write back *

(i) write allocation: 把 block 抓到 cache, 再寫

(ii) write around: 不抓 (出) cache, 直接寫 mem

搭配: write through *

例子?



交大 104

until last time

9. Which of the following statement is correct?

- (a) For write-back cache, both cache and memory will be updated on data-write hit.
- (b) For write-through cache, multiple writes within a block may require only one write to the memory.
- (c) Write-through is widely used in virtual memory in order to keep memory and disk consistent.

- (d) First-level caches are more concerned about hit time, and second-level caches are more concerned about miss rate.
- (e) Increase associativity may increase miss rate due to conflict misses

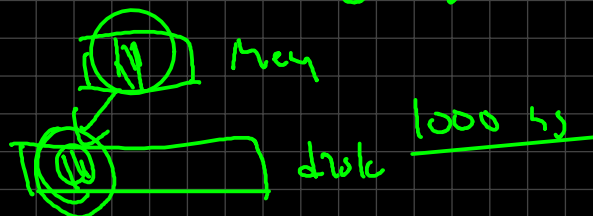
write write, 不同 cache block, write buffer
→ R

cache & mem 同时 update $\rightarrow$ write through

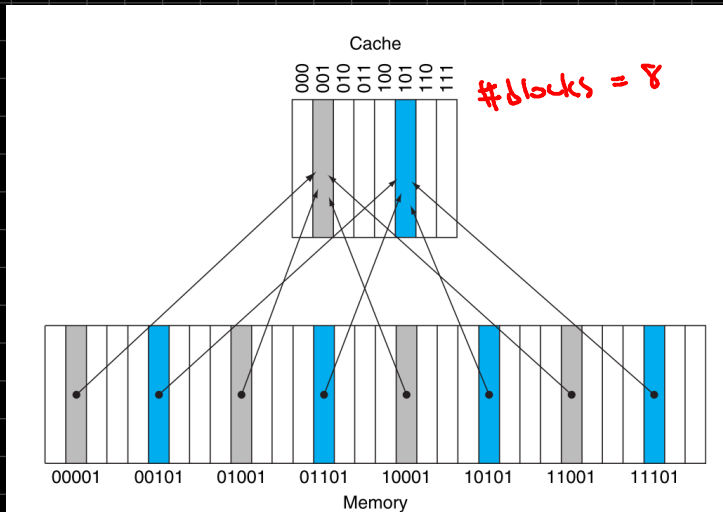
(a) write back 只有在 data write miss 的时候需要 update (cache & memory)

(b) write through 如果要实现累写到一定量再一次性写回 memory (batch)
 $\Rightarrow$ write buffer

(c) write through 不适用 virtual memory, 因为 disk 读取 miss penalty 太高
+ write buffer?



Direct-mapped cache



$\therefore$ 有 $8 = 2^3$ 個 blocks

$\therefore$ 每 address 的 3-bit 做為 Index

ex.

block 5 (101₁₀)

addr. 00101 mod 8 = 101
 01101 mod 8 = 101
 10101 mod 8 = 101
 11101 mod 8 = 101

block (or line) The minimum unit of information that can be either present or not present in a cache.

Step 1: byte address $\rightarrow$ block address (2B cache 的 2B)

ex. block = 4 bytes

byte = 32 bit byte address

address = 0x1111112A

$\downarrow$

block addr. = 0x1111112

Step 2 = (Block address) modulo (Number of blocks in the cache)

ex. # blocks = 8

index = (0x1111112) mod 16^{4²}

= 12 (也就是 map 到 cache 第 12₍₁₀₎ = (0001 0010)₂ 的 block)

Step 3: 多個 mem address 可能映射到相同的 cache block

$\therefore$ 需要額外資訊去 tell 是不同的 mem addr. $\Rightarrow$ tag bit

而 tag 就是除了 index & offset 剩下的 higher-order 部分

(higher order bit)

ex. addr. = 0x 1111 11 2A
 tag index offset

D. Consider 32-bit byte address, a direct-mapped cache of size 64K Bytes with each block of size 4 words. A series of memory accesses occur as follows (assume all states are reset initially): 0XDF105670, 0XDF10567C, 0X23A033A0, 0X23A034A0, 0XDF105678.

23. The number of tag bits is (a) 12 bits, (b) 16 bits, (c) 20 bits, (d) 8 bits.
 24. The number of blocks in the cache is (a) 4K, (b) 2K, (c) 1K, (d) 8K.
 25. The hit access is
 (a) Only 0XDF10567C.
 (b) Only 0X23A034A0.
 (c) 0XDF10567C and 0XDF105678.
 (d) 0XDF10567C, 0X23A034A0 and 0XDF105678.

address = 32 bit

cache size = 64 kB

block size = 4 words

32 x 2 = 1 word = 4 bytes

24. block offset = 2^4 (16 bytes = 4 word), offset = 4 bit

blocks = $\frac{64 \text{ kB}}{4 \times 4 \text{ B}} = 4 \text{ K}$

23. # block = $4 \times 2^{10} = 2^{12}$, index = 12 bit

tag = $32 - 12 - 4 = 16$ bit

25.	tag	index	H/M
0XDF105670	DF10	567	M
✓ 0XDF10567C	DF10	567	H ✓
0X23A033A0	23A0	33A	M
0X23A034A0	23A0	34A	M
✓ 0XDF105678	DF10	567	H ✓

交大 100

2. (20%) Consider three different cache configurations below:

- Cache 1: direct-mapped with one-word blocks.
- Cache 2: direct-mapped with four-word blocks.
- Cache 3: Two-way set associative with two-word blocks and LRU replacement.

Assuming that each cache has a total data size of 16 32-bit words and all of them are initially empty. 20-bit word address is used.

- (a) (3%) What's the size of tag field in one block used in each cache?
 (b) (3%) How many total bits are required for each cache, including tag and valid fields?
 (c) (12%) Given a series of word-address references: 42, 43, 56, 57, 58, 59, 34, 35, 42, and 43. For caches 2 and 3, please label each reference in the list as a hit or a miss (10%), and show the final cache contents (the word addresses in each block or set) after all references (2%).
 (d) (2%) Consider virtual memory system and cache. Please explain alias problem (1%), and the advantage of physically tagged virtually indexed cache (1%).

cache size = 16 word (1 word = 32 bit)

address = 20 bit word address

(a) Cache 1: direct-mapped & 1 word block

blocks = $\frac{16}{1} = 2^4$, index = 4 bit

block offset = 2^0 , offset = 0 bit

tag = $20 - 4 - 0 = 16$ bit

Cache 2: direct-mapped & 4 word block

blocks = $\frac{16}{4} = 2^2$, index = 2 bit

block offset = 2^2 , offset = 2 bit

tag = $20 - 2 - 2 = 16$ bit

cache 3 = 2-way assoc. & 2 word block

$$\# \text{ blocks} = \frac{16}{2} = 2^3$$

$$\# \text{ set} = \frac{2^3}{2} = 2^2, \text{ index} = 2 \text{ bits}$$

$$\text{block offset} = 2^1, \text{ offset} = 1 \text{ bit}$$

$$\text{tag} = 20 - 2 - 1 = 17 \text{ bits}$$

(b)

The total number of bits in a direct-mapped cache is

$$\# \text{ blocks} \times 2^n \times (\text{block size} + \text{tag size} + \text{valid field size}).$$

$$\text{cache 1} = 16 \times (1 + 16 + 32)$$

$$\text{cache 2} = 4 \times (1 + 16 + 4 \times 32)$$

$$\text{cache 3} = (2 \times 4) \times (1 + 17 + 2 \times 32)$$

